

Concept-Aware Lecture Video Retrieval

Project Overview

The **Concept-Aware Lecture Video Retrieval** system is an end-to-end multi-modal search engine designed specifically for educational videos. It allows users to search through long, dense lecture videos using natural language queries and instantly retrieves the exact segments (chunks) of the lecture that contain the requested concept.

Unlike generic video retrieval systems, this project specifically leverages visual clues (on-screen slides/boards), auditory clues (professor's speech), and conceptual drift to segment and retrieve educational content accurately.

Core Technologies & Libraries

The pipeline integrates several state-of-the-art machine learning models to extract multi-modal features:

1. Audio / Speech (ASR):

- **Whisper** (OpenAI): Transcribes spoken audio into text.

2. Visual Text (OCR):

- **EasyOCR**: Extracts textual content directly from the video frames (e.g., text written on whiteboards or projected on slides).

3. Visual Embeddings:

- **CLIP** (OpenAI): Embeds video frames into dense vectors to understand non-textual visual context.

4. Textual Embeddings:

- **Sentence-BERT (all-MiniLM-L6-v2)**: Encodes transcripts, OCR text, and user queries into standard semantic vectors for comparison.

5. Backend & Serving:

- **FastAPI**: Serves the REST API for the frontend and handles asynchronous search requests.
- **Uvicorn**: ASGI web server.

6. Vector Search Engine:

- **FAISS (Facebook AI Similarity Search)**: Enables blazing-fast nearest-neighbor search across thousands of video segment embeddings.
-

The Processing Pipeline (How it works)

The entire backend processing pipeline is triggered by the `scripts/run_full_pipeline.py` script. It consists of six distinct stages:

Stage 1: Feature Extraction

The raw video (.mp4) is sampled (e.g., every 3 seconds). At each step:

- **Speech** is transcribed.
- **Frames** are analyzed for OCR text.
- **Visuals** are passed through CLIP. The output is saved as a compressed .npz file containing time-series data of visual, OCR, and transcript embeddings.

Stage 2: Boundary Detection (Segmentation)

Lectures are rarely uniform. The system needs to cut the video into meaningful "topics" or "segments" rather than arbitrary 1-minute chunks.

- We compute pseudo-labels by measuring sudden spikes in OCR changes (slide transitions) and topic drift in transcripts.
- A **Stage 1 Transformer Model** learns these transitions and outputs confidence scores for segment boundaries.
- The video is then chopped into semantic segments.

Stage 3: Segment Feature Aggregation

For each generated segment, the system pools the embeddings of all frames within that segment into a single cohesive vector representation (one for visual, one for OCR, one for transcript).

Stage 4: Pseudo-Query Generation

To train our retrieval engine, we need (Query, Segment) pairs. The system automatically creates "pseudo-queries" by extracting key phrases from the OCR and transcript text of the segments. This allows self-supervised training without manual labeling.

Stage 5: Cross-Modal Retrieval Training (Stage 2 Model)

A custom **Cross-Modal Retrieval Model** (using Cross-Attention) is trained to map user text queries into the same vector space as the aggregated multi-modal segment features. It learns how to weigh OCR vs. Transcript vs. Visuals based on the query.

Stage 6: Building the Vector Index

Once the model is trained, it generates a final embedding for every segment in the corpus. These embeddings are loaded into a **FAISS Index**. When a user searches, their query is embedded using the same model, and FAISS finds the closest matching segments in milliseconds.

Running the Project

1. Adding New Videos Simply place raw video files (.mp4, .mkv) into the `data/`

raw_videos/ directory.

2. Executing the Pipeline Run the main orchestrator script:

```
python scripts/run_full_pipeline.py
```

Note: The script is incremental. It will automatically detect newly added videos, process them, and rebuild the FAISS index without re-running the heavy extraction on old videos.

3. Starting the Search Server Once the index is built, start the FastAPI server to use the frontend UI:

```
uvicorn app.backend.main:app --host 127.0.0.1 --port 8000
```

Navigate to <http://127.0.0.1:8000> to interact with the search engine.